

Phase 4

Storing Censor Data in Database
(Lennart Esch)

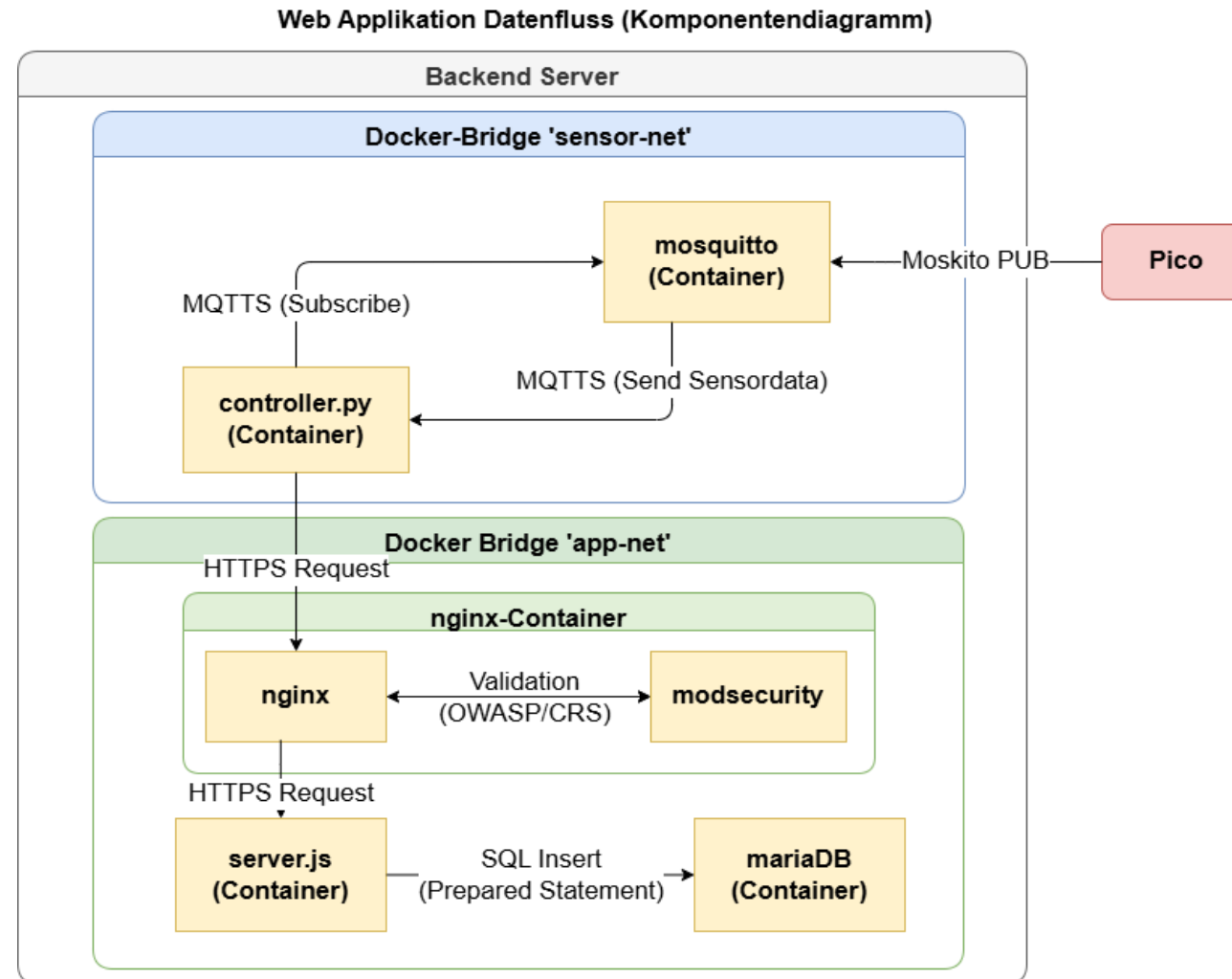
Agenda

1. Problemstellung
2. Deployment
3. Aufgaben
4. Controller
5. NGINX
6. WebServer
7. MariaDB
8. Zusammenspiel

Problemstellung

- Datentransfer
 - MQTT-Broker -> Datenbank
 - Sichere Übertragung
- Kommunikation
 - PICO <-> Controller

Deployment



Deployment

- Mittels docker-compose.yml

```
pi@Backend:~/Anwendungsprojekt.Inf $ sudo docker compose up
[+] up 5/5
✓Container mqtt Recreated
✓Container mariadb Recreated
✓Container backend Recreated
✓Container nginx_proxy Recreated
✓Container anwendungsprojektinf-controller-1 Recreated
```

```
services:
  webapp:
    container_name: backend
    # ...

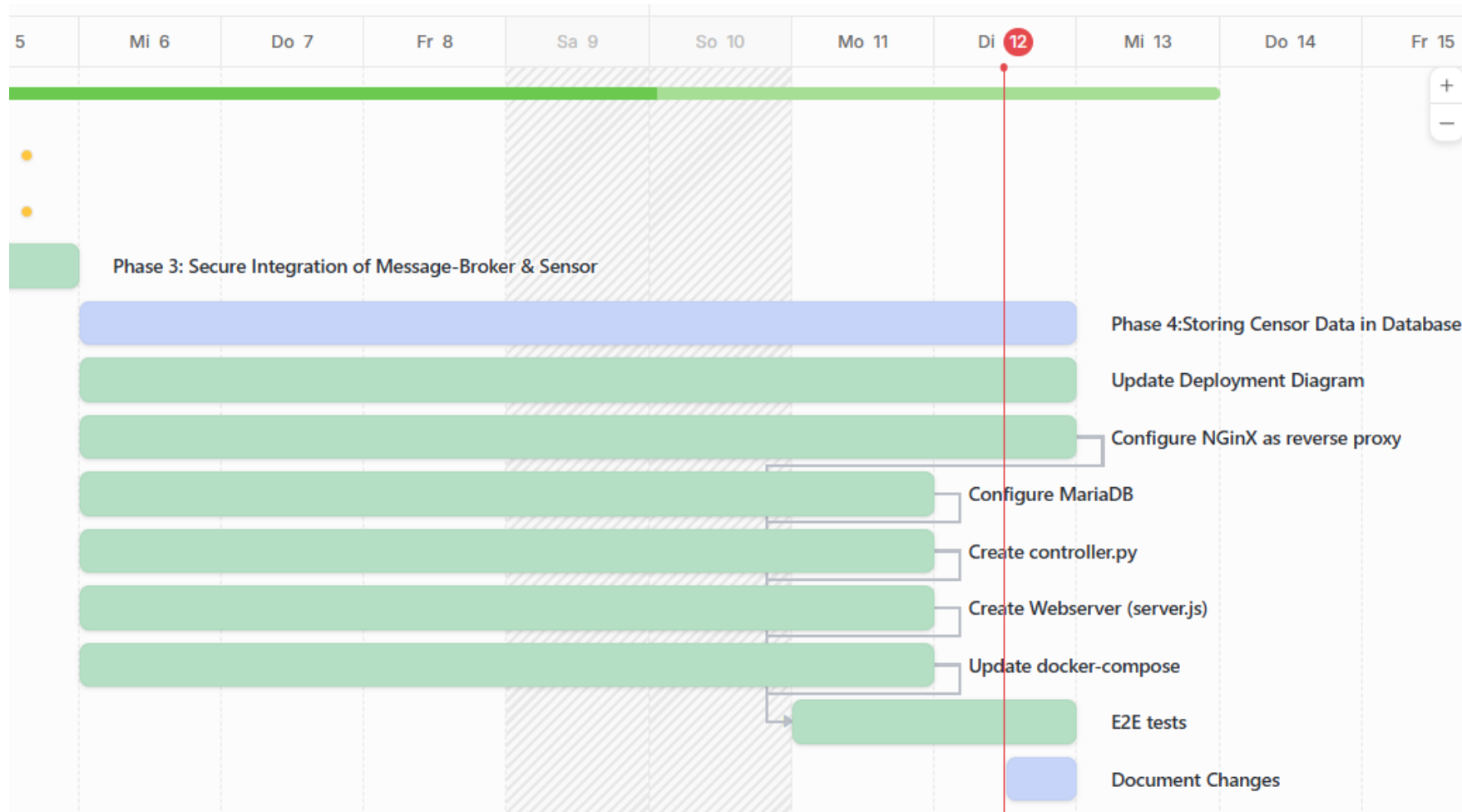
  db:
    container_name: mariadb
    # ...

  mqtt:
    container_name: mqtt
    # ...

  nginx:
    container_name: nginx_proxy
    # ...

  controller:
    container_name: pico_controller
    # ...
```

Aufgaben



Barnabas Steiner - Benjamin Hager - Jannis Weber - Lennart
Esch - Tim Dorozynski - Walter Neer

Controller

- Python Script
- Image gebaut durch Dockerfile
- Dateizugriff auf Zertifikat
- HTTPS Zugriff auf host-system

```
controller:
  build: sensor-net/
  restart: unless-stopped
  volumes:
    - ./sensor-data/ca.crt:/ca.crt:ro
  depends_on:
    - mqtt
    - nginx
  extra_hosts:
    - "www.lab.local:host-gateway"
  networks:
    sensor-network:
```

Controller

- Basierend auf Python 3.12 Image
- Umgebungsvariablen für das Skript
- Installation aus requirements.txt
 - Paho_mqtt
 - Requests

```
FROM python:3.12-alpine
WORKDIR /code
ENV MQTT_HOST=192.168.4.18
ENV MQTT_PORT=8883
ENV MQTT_USER=testuser
ENV MQTT_PASSWORD=test
ENV MQTT_TOPIC=sensor/data
ENV MQTT_CA_FILE=ca.crt
ENV BACKEND_URL=https://www.lab.local/api/internal/sensordata
ENV API_KEY=api_key
ENV MQTT_TLS_INSECURE=false
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python3", "controller.py"]
```


Controller

- Client aus paho_mqtt
- Empfang und Validierung
- Weiterleiten an Nginx mit <https://www.lab.local/api/internal/sensordata>

```
def on_message(client, userdata, msg): 1 usage 2 Jannis *
    log.debug("Empfangen auf %s: %s", msg.topic, msg.payload)
    parsed = parse_and_validate(msg.payload)
    if parsed is None:
        return
    temperature, humidity = parsed
    # MQTT-Payload enthaelt keinen Timestamp -> beim Empfang setzen
    timestamp = datetime.now(timezone.utc).isoformat(timespec="seconds")
    forward_to_backend(temperature, humidity, timestamp)
```

```
client = mqtt.Client(
    client_id=MQTT_CLIENT_ID,
    callback_api_version=mqtt.CallbackAPIVersion.VERSION2,
)

if MQTT_USER:
    client.username_pw_set(MQTT_USER, MQTT_PASSWORD)

# TLS-Kontext: Server-Zertifikat gegen die Projekt-CA verifizieren
ssl_ctx = ssl.create_default_context(cafile=MQTT_CA_FILE)
if MQTT_TLS_INSECURE:
    ssl_ctx.check_hostname = False
    ssl_ctx.verify_mode = ssl.CERT_NONE
    log.warning(
        "TLS-Verifizierung DEAKTIVIERT (MQTT_TLS_INSECURE=true) - "
        "nur fuer Tests verwenden!"
    )
client.tls_set_context(ssl_ctx)

client.on_connect = on_connect
client.on_disconnect = on_disconnect
client.on_message = on_message
```

Nginx

- OWASP-Image
 - SQL Injection
 - XSS
 - Directory Traversal
- HTTP/HTTPS Ports geöffnet
- Niedrige Sicherheitsstufe
- Zugriff auf Zertifikate

```
nginx:
  image: owasp/modsecurity-crs:nginx
  container_name: nginx_proxy
  restart: unless-stopped
  ports:
    - "443:443"
    - "80:80"
  environment:
    - PARANOIA=1
    - BACKEND=https://webapp:3000
    - MSG="ModSecurity Attack Detected"
  volumes:
    - ./nginx/certs/www_key.key:/etc/nginx/certs/www_key.key:ro
    - ./nginx/certs/www_cert.cert:/etc/nginx/certs/www_cert.cert:ro
    - ./nginx/nginx.conf:/etc/nginx/templates/nginx.conf.template:ro
  networks:
    app-network:
  depends_on:
    - webapp
```

Nginx

- HTTPS Weiterleitung auf webapp
- Selbsterstellte Zertifikate

```
server {  
    listen 443 ssl;  
    server_name www.lab.local;  
  
    ssl_certificate /etc/nginx/certs/www_cert.cert;  
    ssl_certificate_key /etc/nginx/certs/www_key.key;  
  
    ssl_protocols TLSv1.2 TLSv1.3;  
    ssl_ciphers HIGH:!aNULL:!MD5;  
  
    location / {  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_cache_bypass $http_upgrade;  
  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
  
        proxy_pass http://webapp:3000;  
    }  
}
```

Webserver

- Node/Express server
- Image gebaut aus Dockerfile
- Datenbank Credentials
- Api-Key

```
webapp:
  build: .
  container_name: backend
  restart: always
  environment:
    - DB_HOST=mariadb
    - DB_USER=websrv_write
    - DB_PASSWORD=pass-0123-iot
    - DB_NAME=myapp
    - PORT=3000
    - API_KEY=api_key
  depends_on:
    db:
      condition: service_healthy
  networks:
    - app-network
```

Webserver

- Basiert auf Node-Image
- Installiert mit package.json
 - Express
 - Dotenv
 - MariaDB
- Läuft intern auf 3000

```
FROM node:18-alpine

WORKDIR /usr/src/app

COPY package*.json ./
RUN npm install --production

COPY . .

EXPOSE 3000

CMD [ "node", "app-net/server.js" ]
```

Webserver

```
app.post('/api/internal/sensordata', authenticateApiKey, async (req, res) : Promise<void> => {  ⚡ DragoBenny +1
  // validate data
  const {temperature, humidity, timestamp} = validateSensorPayload(req.body);

  let conn;
  try {
    if (!temperature) throw Error("Denied: ")
    const pool = await getPool();
    conn = await pool.getConnection();
    console.log("Connection to mariaDB established")

    // Prepared Statement
    const sql :string = "INSERT INTO sensor_data (temperature, humidity, timestamp) VALUES (?, ?, ?)";
    const result = await conn.query(sql, [temperature, humidity, timestamp]);

    console.log("Execution: ", result)
    res.status(201).json({
      status: "ok"
    });
  } catch (err) {
    res.status(400).json({error: err.message || "invalid payload"});
  } finally {
    if (conn) conn.release();
  }
}
```

MariaDB

- Event Scheduler
- Credentials
- Persistenz
- Healthcheck

```
db:
  image: mariadb:latest
  command: --event-scheduler=ON
  container_name: mariadb
  restart: always
  environment:
    - MARIADB_ROOT_PASSWORD=root_password
    - MARIADB_DATABASE=myapp
    - MARIADB_USER=user
    - MARIADB_PASSWORD=password
  volumes:
    - /mariadb/db:/var/lib/mysql
    - /mariadb/conf.d:/etc/mysql/conf.d:ro
    - /mariadb/01_config.sql:/docker-entrypoint-initdb.d/01_config.sql:ro
  healthcheck:
    test: [ "CMD", "healthcheck.sh", "--connect", "--innodb_initialized" ]
    start_period: 20s
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    app-network:
```

MariaDB

```
CREATE DATABASE IF NOT EXISTS myapp;

USE myapp;

CREATE TABLE IF NOT EXISTS sensor_data (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    timestamp   TIMESTAMP      NOT NULL,
    temperature DECIMAL(4,1)   NOT NULL,
    humidity    DECIMAL(4,1) NOT NULL
);

CREATE TABLE IF NOT EXISTS sensor_data_archive (
    id          INT PRIMARY KEY,
    timestamp   TIMESTAMP      NOT NULL,
    temperature DECIMAL(4,1)   NOT NULL,
    humidity    DECIMAL(4,1) NOT NULL
);
```

```
DELIMITER $$

CREATE EVENT IF NOT EXISTS archive_old_sensor_data
    ON SCHEDULE EVERY 1 DAY
    STARTS CURRENT_TIMESTAMP
    DO
    BEGIN
        INSERT INTO sensor_data_archive (id, timestamp, temperature, humidity)
            SELECT id, timestamp, temperature, humidity
            FROM sensor_data
            WHERE timestamp < NOW() - INTERVAL 7 DAY;

        DELETE FROM sensor_data
            WHERE timestamp < NOW() - INTERVAL 7 DAY;
    END$$

DELIMITER ;
```


Zusammenspiel

- E2E Test

```
controller-1 | /usr/local/lib/python3.12/site-packages/urllib3/connectionpool.py:1110: InsecureRequestWarning: Unverified HTTPS request is being made to host 'www.lab.local'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/advanced-usage.html#tls-warnings
controller-1 | warnings.warn(
backend      | Connection to mariaDB established
backend      | Execution: OkPacket { affectedRows: 1, insertId: 610n, warningStatus: 0 }
nginx_proxy  | 172.18.0.1 - - [12/May/2026:14:27:16 +0000] "POST /api/internal/sensordata HTTP/1.1" 201 15 "-" "python-requests/2.33.1"
controller-1 | 2026-05-12 14:27:16,781 [INFO] controller: An Backend weitergeleitet: {'temperature': 23.0, 'humidity': 26.8, 'timestamp': '2026-05-12T14:27:16+00:00'} (status 201)
```

- Warnung Zertifikat Nginx
- Datenbank Query Ausführung
- Response Code 201 an Controller

Zusammenspiel

- Verbindung mit MariaDB
- Database myapp
- Sensor Data wird gefüllt

```
sudo docker exec -it mariadb mariadb -u root -proot_password
```

```
MariaDB [(none)]> use myapp;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup:  
  
Database changed  
MariaDB [myapp]> SELECT * FROM sensor_data;
```

id	timestamp	temperature	humidity
1	2026-05-12 07:31:19	22.3	28.4
2	2026-05-12 07:32:00	22.3	29.7
3	2026-05-12 07:32:42	22.3	29.1
4	2026-05-12 07:33:24	22.3	28.1
5	2026-05-12 07:34:05	22.2	27.3
6	2026-05-12 07:34:47	22.2	27.4

Vielen Dank für Ihre Aufmerksamkeit

Barnabas Steiner - Benjamin Hager - Jannis Weber - Lennart
Esch - Tim Dorozynski - Walter Neer